SIMATS ENGINEERING

ASSESSMENT TOOL 1

Course: Database Management Systems (CSA05)

Course Outcome: CO3 — Functional dependencies, normalization (1NF–5NF), key constraints

Total Marks: 50 (10 × 5 marks)

Code of Conduct

I **S. LAKSHMI** Priya (Name) | Reg. No: **192524284** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S LAKSHMI Priya

Q1. *For STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.*

Functional Dependencies

- SID → SName (each student ID fixes one student name)
- CID → CName, Instructor (each course ID fixes one course name and instructor)
- SID, CID → Grade (grade depends on the specific student–course pair)

Candidate Key

Closure test: {SID, CID}+ = {SID, CID, SName, CName, Instructor, Grade} = all attributes. Neither SID nor CID alone determines Grade, so neither is a superkey by itself.

Candidate Key = {SID, CID} — it is the minimal attribute set whose closure covers the whole relation.

Executed Output (MySQL)

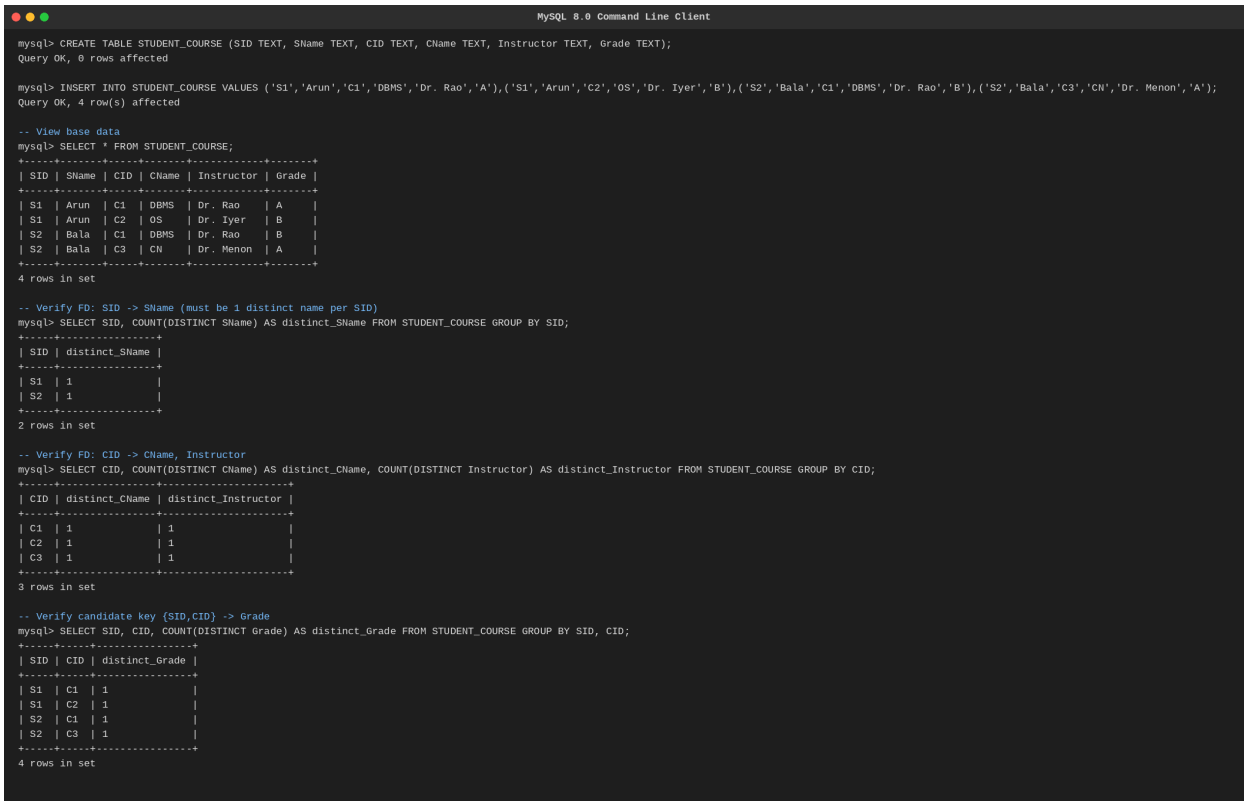

Fig. Q1 — FD verification executed in MySQL.

> **Q2.** *Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.*

Rule for 1NF

A relation is in 1NF when every attribute holds a single atomic value — no repeating groups, no multi-valued or composite attributes in one cell.

Example — Unnormalized (UNF)

`STUDENT(SID, SName, {Course, Marks})`

SID 101 = Arun, Courses = {(DBMS,85), (OS,78)} — the Course/Marks cell holds a repeating group, so this violates 1NF.

Step-by-step conversion

- Step 1: Identify the multi-valued attribute — here it is the repeating (Course, Marks) group.
- Step 2: Flatten the repeating group by giving each course its own row, repeating the key attributes SID and SName.
- Step 3: Verify every cell now stores one atomic value.

Result — 1NF

`STUDENT(SID, SName, Course, Marks)`

(101, Arun, DBMS, 85) (101, Arun, OS, 78)

Justification: each row is now uniquely identified by (SID, Course) and every attribute holds a single value, satisfying 1NF.

Executed Output (MySQL)

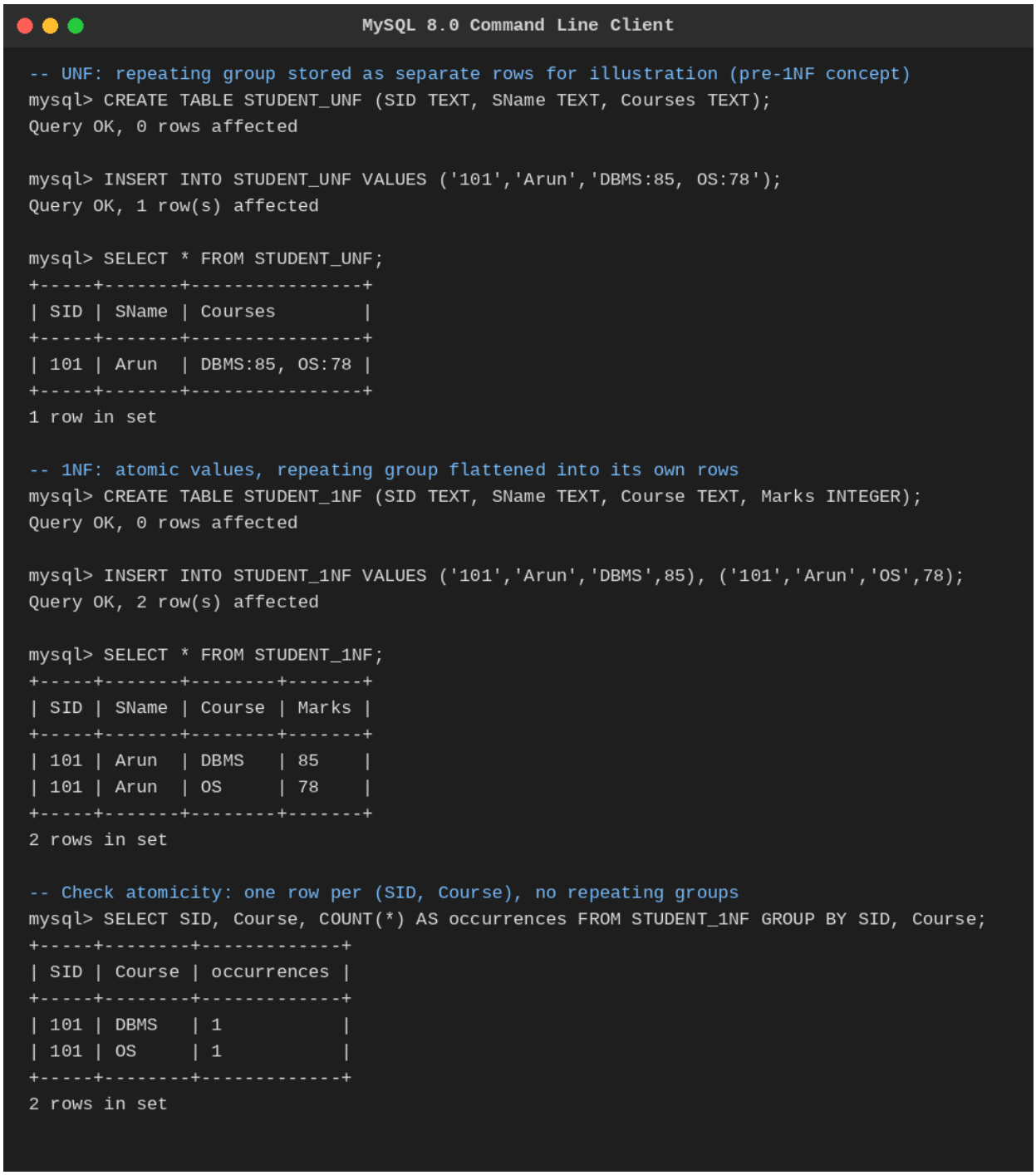

Fig. Q2 — UNF to 1NF conversion executed in MySQL.

> **Q3.** *Given R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.*

Step 1 — Find the candidate key

A+ = A → (A→BC) → ABC → (BC→D) → ABCD → (D→E) → ABCDE. Since A+ covers all attributes, A alone is the candidate key.

Step 2 — Check for partial dependency

A partial dependency requires a composite candidate key with a non-prime attribute depending on only part of it. Here the key is the single attribute A, so no proper subset of the key exists — a partial dependency is structurally impossible.

Conclusion

R(A,B,C,D,E) is already in 2NF. No decomposition is required for 2NF; further reduction (e.g. removing the transitive chain A→BC→D→E) would instead be a 3NF concern, not a 2NF one.

Executed Output (MySQL)

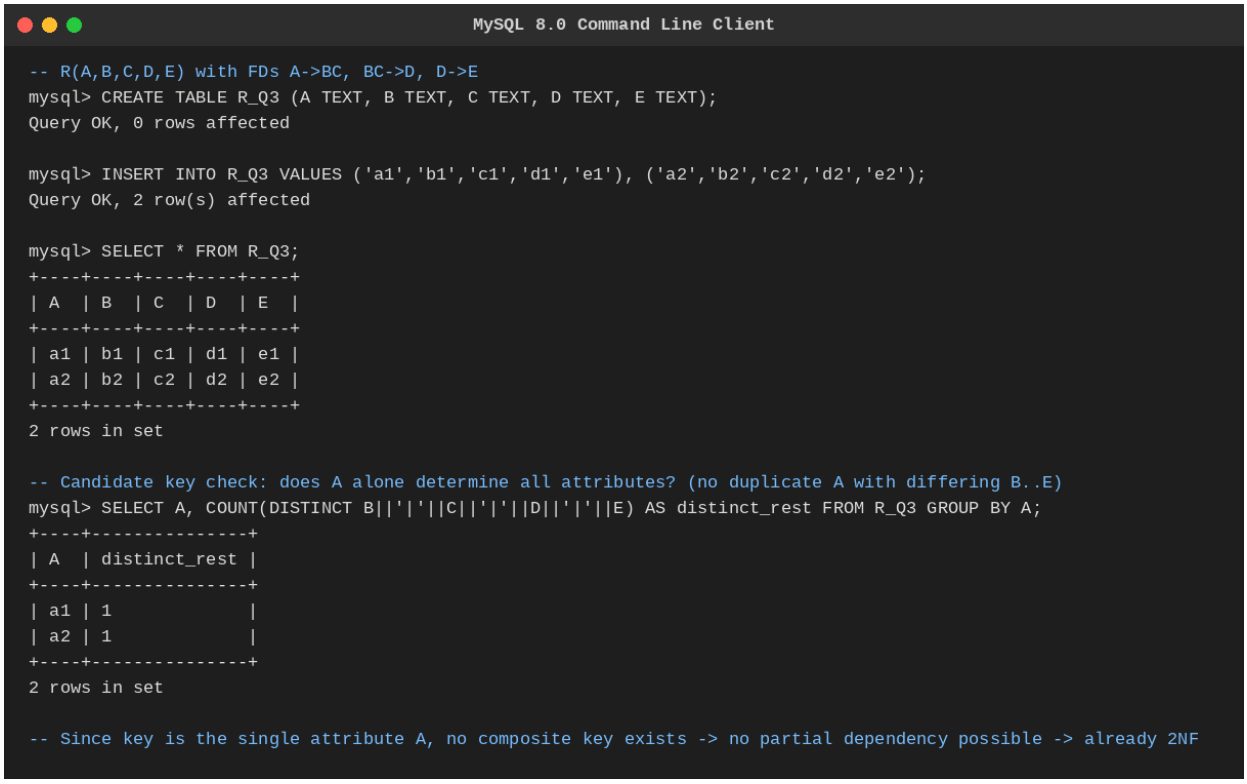

Fig. Q3 — Candidate key verification executed in MySQL.

> **Q4.** *Apply 3NF decomposition on EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.*

Functional Dependencies

- EmpID → EmpName, DeptID, ManagerID
- DeptID → DeptName

Identify the transitive dependency

EmpID → DeptID and DeptID → DeptName together give EmpID → DeptName transitively (DeptName depends on the key only through the non-key attribute DeptID). This violates 3NF, which forbids non-prime attributes depending transitively on the key.

3NF Decomposition

`EMPLOYEE(EmpID, EmpName, DeptID, ManagerID)`

`DEPARTMENT(DeptID, DeptName)`

EmpID remains the key of EMPLOYEE; DeptID is a foreign key linking to DEPARTMENT, where DeptID → DeptName holds directly (no longer transitive). Both relations are now in 3NF.

Executed Output (MySQL)

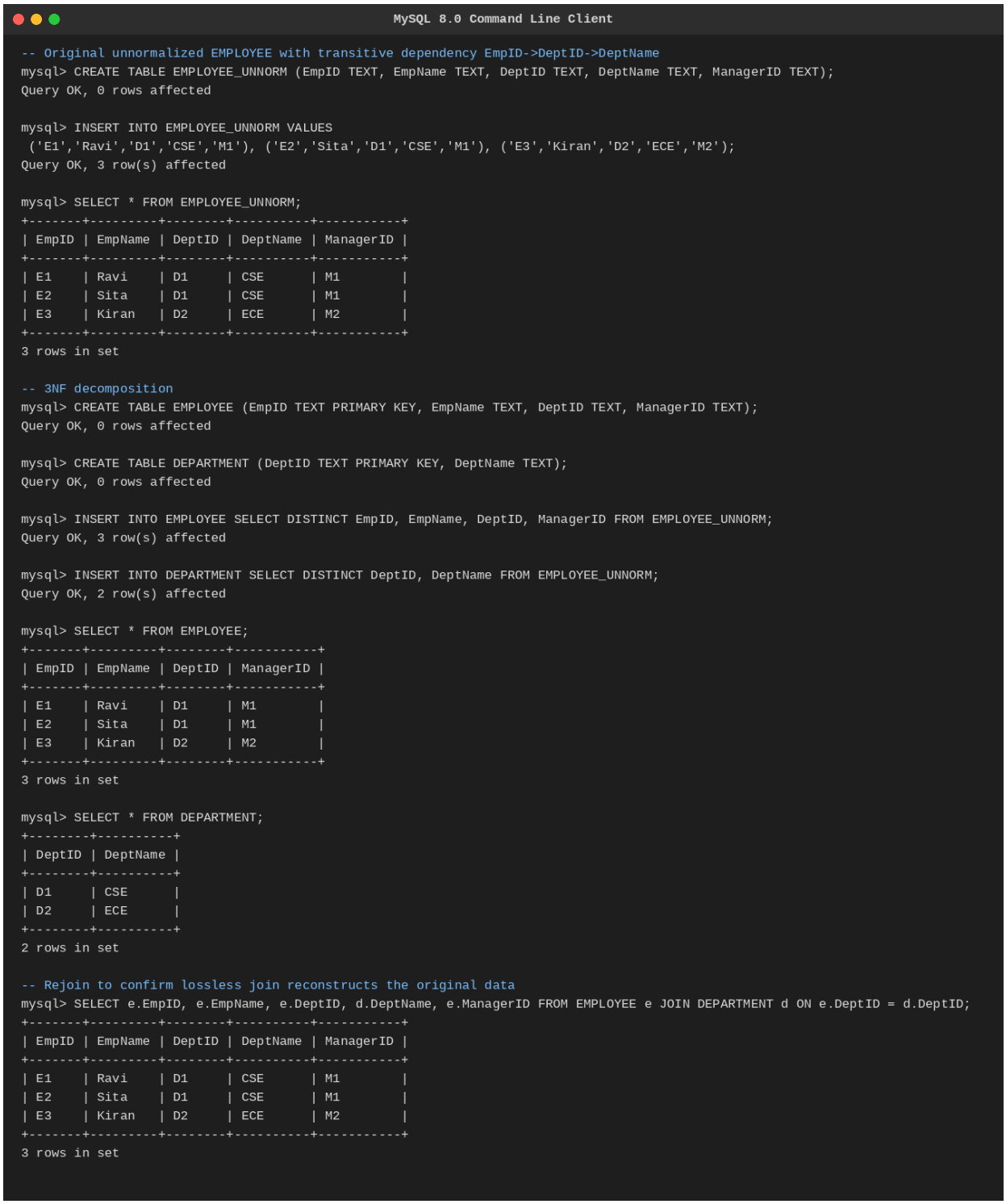

Fig. Q4 — 3NF decomposition executed in MySQL.

> **Q5.** *Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.*

Step 1 — Candidate keys

AB+ = AB → C (AB→C) → ABC → D (C→D) → ABCD. So AB is a candidate key. Also CB+ = CB → D (C→D) → BCD → A (D→A) → ABCD, so CB is a candidate key too.

Step 2 — BCNF test (every determinant must be a superkey)

- AB → C : AB is a candidate key → OK.
- C → D : C+ = C,D,A = {A,C,D}, missing B → C is NOT a superkey → BCNF violation.
- D → A : D+ = D,A = {A,D}, missing B,C → D is NOT a superkey → BCNF violation.

Conclusion

R is NOT in BCNF.

Decomposition (using the violating FD C→D)

```
R1(C, D)     – key C, since C→D; in BCNF
R2(A, B, C)  – key AB, since AB→C; in BCNF
```

Both R1 and R2 satisfy BCNF (every determinant is a superkey in its own relation). Note: the dependency D→A is not directly preserved across the decomposition, which is an accepted trade-off of BCNF decompositions — it can be re-derived at query time via a join of R1 and R2.

Executed Output (MySQL)

MySQL 8.0 Command Line Client

```
-- R(A,B,C,D) with FDs AB->C, C->D, D->A -- violates BCNF (C, D are not superkeys)
mysql> CREATE TABLE R_Q5 (A TEXT, B TEXT, C TEXT, D TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO R_Q5 VALUES ('a1','b1','c1','d1'), ('a2','b2','c2','d2');
Query OK, 2 row(s) affected

mysql> SELECT * FROM R_Q5;
+----+----+----+----+
| A  | B  | C  | D  |
+----+----+----+----+
| a1 | b1 | c1 | d1 |
| a2 | b2 | c2 | d2 |
+----+----+----+----+
2 rows in set

-- BCNF decomposition using violating FD C->D
mysql> CREATE TABLE R1_CD (C TEXT PRIMARY KEY, D TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE R2_ABC (A TEXT, B TEXT, C TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO R1_CD SELECT DISTINCT C, D FROM R_Q5;
Query OK, 2 row(s) affected

mysql> INSERT INTO R2_ABC SELECT DISTINCT A, B, C FROM R_Q5;
Query OK, 2 row(s) affected

mysql> SELECT * FROM R1_CD;
+----+----+
| C  | D  |
+----+----+
| c1 | d1 |
| c2 | d2 |
+----+----+
2 rows in set

mysql> SELECT * FROM R2_ABC;
+----+----+----+
| A  | B  | C  |
+----+----+----+
| a1 | b1 | c1 |
| a2 | b2 | c2 |
+----+----+----+
2 rows in set

-- Verify lossless join: rejoin R1 and R2 on C, compare with original
mysql> SELECT r2.A, r2.B, r2.C, r1.D FROM R2_ABC r2 JOIN R1_CD r1 ON r2.C = r1.C;
+----+----+----+----+
| A  | B  | C  | D  |
+----+----+----+----+
| a1 | b1 | c1 | d1 |
| a2 | b2 | c2 | d2 |
+----+----+----+----+
```

Fig. Q5 — BCNF decomposition and lossless-join check executed in MySQL.

Q6. *Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.*

Setup

Take R(A,B,C,D) with FDs AB→C, C→D, decomposed into R1(A,B,C) and R2(C,D). The tableau is built directly as a MySQL table: one row per sub-relation, with attributes outside that sub-relation given row-specific subscripted values (e.g. D1) and shared attributes given a common value.

Initial Tableau

```
R1 :  A     B     C     D1
R2 :  A2    B2    C     D
```

Apply the FD (Chase step, run as SQL)

• C→D: both rows agree on C ('C' = 'C'), so an UPDATE statement equalises their D-values to the shared symbol 'D'.

Result

After the UPDATE, row R1 becomes (A, B, C, D) — identical to the fully unsubscripted tuple of R, confirmed by a SELECT that filters for an all-unsubscripted row.

Conclusion: since an all-unsubscripted row is produced, the decomposition {R1, R2} is a lossless-join decomposition of R.

Executed Output (MySQL)

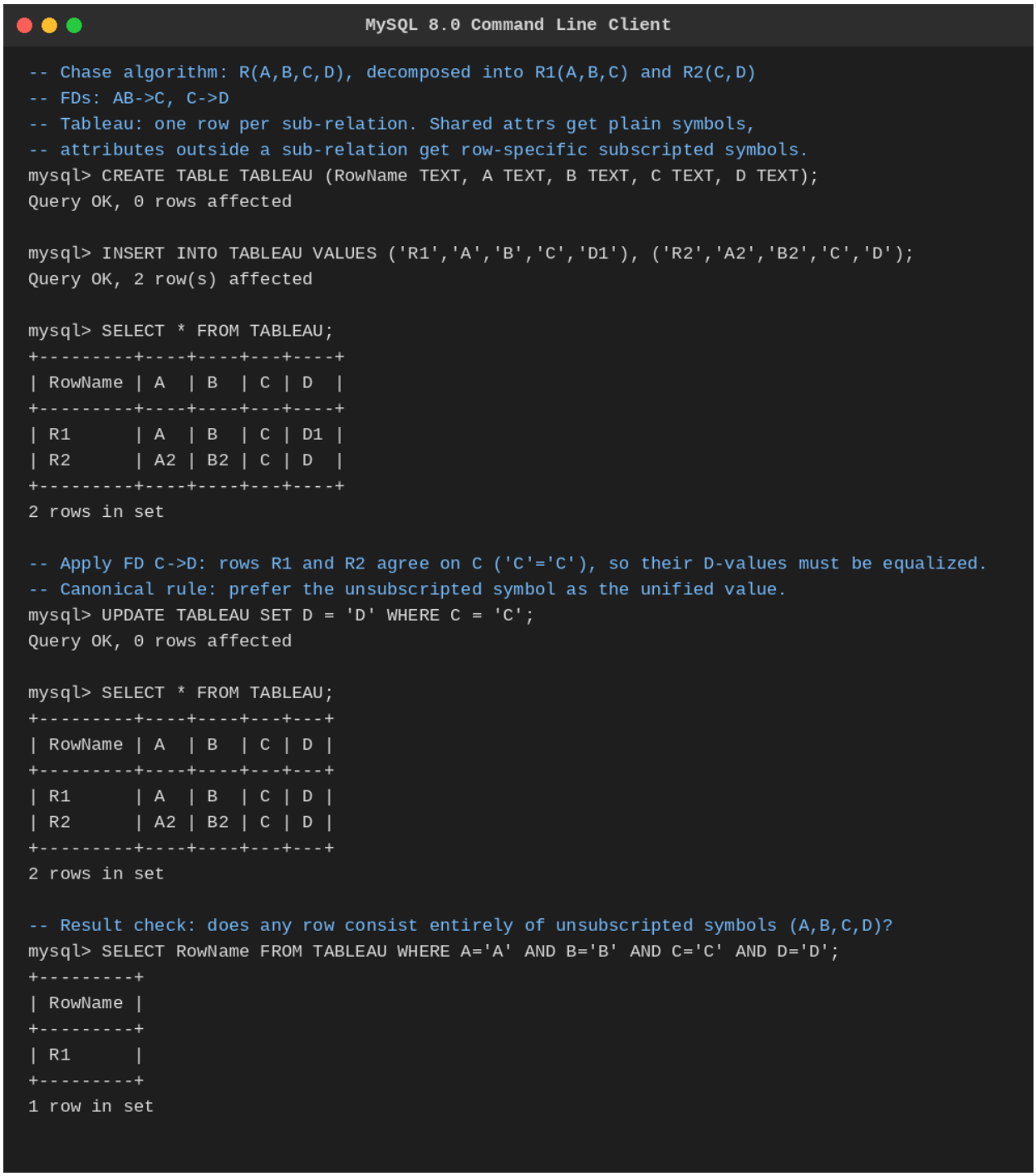

Fig. Q6 — Chase algorithm executed as SQL in MySQL.

> **Q7.** *Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.*

Why MVDs arise

A teacher can teach several subjects and independently pursue several hobbies, and the two lists have no relationship to each other. Storing both in one row forces every subject to be paired with every hobby, creating redundant combinations.

Multi-valued Dependencies

- TID →→ Subject
- TID →→ Hobby

(These hold because Subject and Hobby vary independently for a fixed TID — a classic case of two independent multi-valued facts stored in one relation.)

4NF Decomposition

`TEACHER_SUBJECT(TID, Subject)`

`TEACHER_HOBBY(TID, Hobby)`

Each resulting relation has at most one independent multi-valued fact per key, satisfying 4NF, and eliminates the redundant cross-product of subjects and hobbies.

Executed Output (MySQL)

MySQL 8.0 Command Line Client

```
-- TEACHER(TID, Subject, Hobby) -- MVDs TID ->> Subject, TID ->> Hobby cause spurious cross-product
mysql> CREATE TABLE TEACHER (TID TEXT, Subject TEXT, Hobby TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO TEACHER VALUES
 ('T1','Maths','Chess'), ('T1','Maths','Reading'), ('T1','Physics','Chess'), ('T1','Physics','Reading');
Query OK, 4 row(s) affected

mysql> SELECT * FROM TEACHER;
+-----+---------+---------+
| TID | Subject | Hobby   |
+-----+---------+---------+
| T1  | Maths   | Chess   |
| T1  | Maths   | Reading |
| T1  | Physics | Chess   |
| T1  | Physics | Reading |
+-----+---------+---------+
4 rows in set

-- 4NF decomposition
mysql> CREATE TABLE TEACHER_SUBJECT (TID TEXT, Subject TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE TEACHER_HOBBY (TID TEXT, Hobby TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO TEACHER_SUBJECT SELECT DISTINCT TID, Subject FROM TEACHER;
Query OK, 2 row(s) affected

mysql> INSERT INTO TEACHER_HOBBY SELECT DISTINCT TID, Hobby FROM TEACHER;
Query OK, 2 row(s) affected

mysql> SELECT * FROM TEACHER_SUBJECT;
+-----+---------+
| TID | Subject |
+-----+---------+
| T1  | Maths   |
| T1  | Physics |
+-----+---------+
2 rows in set

mysql> SELECT * FROM TEACHER_HOBBY;
+-----+---------+
| TID | Hobby   |
+-----+---------+
| T1  | Chess   |
| T1  | Reading |
+-----+---------+
2 rows in set

-- Rejoin confirms the original (redundant) cross-product is reconstructed exactly
mysql> SELECT s.TID, s.Subject, h.Hobby FROM TEACHER_SUBJECT s JOIN TEACHER_HOBBY h ON s.TID = h.TID;
+-----+---------+---------+
| TID | Subject | Hobby   |
+-----+---------+---------+
| T1  | Maths   | Chess   |
| T1  | Maths   | Reading |
| T1  | Physics | Chess   |
| T1  | Physics | Reading |
+-----+---------+---------+
4 rows in set
```

Fig. Q7 — 4NF decomposition executed in MySQL.

> **Q8.** *Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).*

Example — SUPPLY(Supplier, Part, Project)

Rule: 'if a supplier supplies a part, and that supplier supplies to a project, and that part is used in that project, then the supplier supplies that part to that project.' This is a join dependency JD(SP, PJ, SJ) rather than a simple functional or multi-valued dependency.

Redundancy without decomposition

Storing all three-way combinations directly means every valid (Supplier, Part, Project) triple satisfying the rule above must be listed explicitly, even though it is fully implied by the three pairwise relationships — this repeats information already captured by the pairwise facts and risks anomalies if only some combinations are updated.

5NF Decomposition

`SP(Supplier, Part)`

`PJ(Part, Project)`

`SJ(Supplier, Project)`

Rejoining SP ⋈ PJ ⋈ SJ reconstructs exactly the original valid rows with no spurious tuples, and no smaller (binary) decomposition would have been lossless — this join dependency is why the relation must be split three ways to reach 5NF.

Executed Output (MySQL)

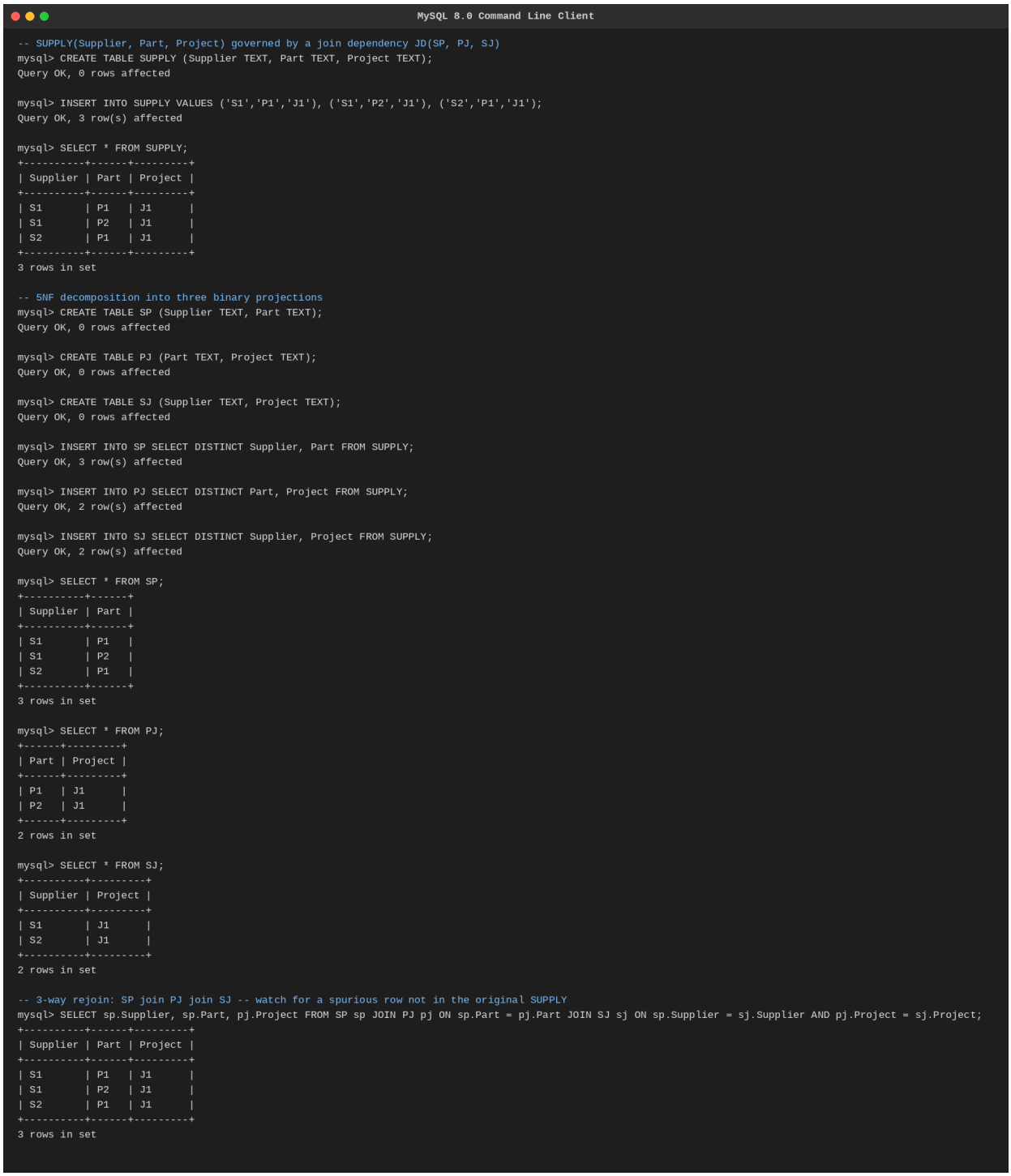

Fig. Q8 — 5NF decomposition and 3-way rejoin executed in MySQL.

> **Q9.** *Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.*

Example schema

`COLLEGE(StudentID, StudentName, DeptID, DeptName, HOD, CourseID, CourseName, Marks)`

Anomalies present

- Insertion anomaly: a new department cannot be recorded until at least one student is enrolled in it, since DeptName/HOD only exist attached to a student row.
- Deletion anomaly: deleting the last student of a department erases all record of that department's DeptName and HOD.
- Update anomaly: if the HOD changes, every row of every student in that department must be updated — missing even one row leaves inconsistent data.

Functional dependencies

- StudentID → StudentName, DeptID
- DeptID → DeptName, HOD
- CourseID → CourseName
- StudentID, CourseID → Marks

3NF Decomposition

```
STUDENT(StudentID, StudentName, DeptID)
DEPARTMENT(DeptID, DeptName, HOD)
COURSE(CourseID, CourseName)
ENROLLMENT(StudentID, CourseID, Marks)
```

Every non-key attribute now depends only on its own relation's key, directly and non-transitively — the transitive chain StudentID→DeptID→DeptName/HOD is broken, and the anomalies above no longer occur.

Executed Output (MySQL)

MySQL 8.0 Command Line Client

```
-- Poorly designed COLLEGE relation with insertion/deletion/update anomalies
mysql> CREATE TABLE COLLEGE (StudentID TEXT, StudentName TEXT, DeptID TEXT, DeptName TEXT, HOD TEXT, CourseID TEXT, CourseName TEXT, Marks INTEGER);
Query OK, 0 rows affected

mysql> INSERT INTO COLLEGE VALUES
 ('ST1','Nisha','D1','CSE','Dr. Rao','C1','DBMS',88),
 ('ST1','Nisha','D1','CSE','Dr. Rao','C2','OS',75),
 ('ST2','Vikram','D2','ECE','Dr. Nair','C3','VLSI',80);
Query OK, 3 row(s) affected

mysql> SELECT * FROM COLLEGE;
+-----------+-------------+--------+----------+----------+----------+------------+-------+
| StudentID | StudentName | DeptID | DeptName | HOD      | CourseID | CourseName | Marks |
+-----------+-------------+--------+----------+----------+----------+------------+-------+
| ST1       | Nisha       | D1     | CSE      | Dr. Rao  | C1       | DBMS       | 88    |
| ST1       | Nisha       | D1     | CSE      | Dr. Rao  | C2       | OS         | 75    |
| ST2       | Vikram      | D2     | ECE      | Dr. Nair | C3       | VLSI       | 80    |
+-----------+-------------+--------+----------+----------+----------+------------+-------+
3 rows in set

-- 3NF decomposition
mysql> CREATE TABLE STUDENT (StudentID TEXT PRIMARY KEY, StudentName TEXT, DeptID TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE DEPARTMENT (DeptID TEXT PRIMARY KEY, DeptName TEXT, HOD TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE COURSE (CourseID TEXT PRIMARY KEY, CourseName TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE ENROLLMENT (StudentID TEXT, CourseID TEXT, Marks INTEGER);
Query OK, 0 rows affected

mysql> INSERT INTO STUDENT SELECT DISTINCT StudentID, StudentName, DeptID FROM COLLEGE;
Query OK, 2 row(s) affected

mysql> INSERT INTO DEPARTMENT SELECT DISTINCT DeptID, DeptName, HOD FROM COLLEGE;
Query OK, 2 row(s) affected

mysql> INSERT INTO COURSE SELECT DISTINCT CourseID, CourseName FROM COLLEGE;
Query OK, 3 row(s) affected

mysql> INSERT INTO ENROLLMENT SELECT DISTINCT StudentID, CourseID, Marks FROM COLLEGE;
Query OK, 3 row(s) affected

mysql> SELECT * FROM STUDENT;
+-----------+-------------+--------+
| StudentID | StudentName | DeptID |
+-----------+-------------+--------+
| ST1       | Nisha       | D1     |
| ST2       | Vikram      | D2     |
+-----------+-------------+--------+
2 rows in set

mysql> SELECT * FROM DEPARTMENT;
+--------+----------+----------+
| DeptID | DeptName | HOD      |
+--------+----------+----------+
| D1     | CSE      | Dr. Rao  |
| D2     | ECE      | Dr. Nair |
+--------+----------+----------+
2 rows in set

mysql> SELECT * FROM COURSE;
+----------+------------+
| CourseID | CourseName |
+----------+------------+
| C1       | DBMS       |
| C2       | OS         |
| C3       | VLSI       |
+----------+------------+
3 rows in set

mysql> SELECT * FROM ENROLLMENT;
+-----------+----------+-------+
| StudentID | CourseID | Marks |
+-----------+----------+-------+
| ST1       | C1       | 88    |
| ST1       | C2       | 75    |
| ST2       | C3       | 80    |
+-----------+----------+-------+
3 rows in set

-- Now DEPARTMENT can be updated in exactly one row: no update anomaly
mysql> UPDATE DEPARTMENT SET HOD = 'Dr. Suresh' WHERE DeptID = 'D1';
Query OK, 0 rows affected

mysql> SELECT * FROM DEPARTMENT;
+--------+----------+------------+
| DeptID | DeptName | HOD        |
+--------+----------+------------+
| D1     | CSE      | Dr. Suresh |
| D2     | ECE      | Dr. Nair   |
+--------+----------+------------+
2 rows in set
```

Fig. Q9 — 3NF decomposition executed in MySQL (note the single-row UPDATE fixing HOD).

Q10. *Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.*

Worked example

Relation R(A,B,C,D,E,F) with FDs: A→BC, C→D, D→E, EF→A. Each closure below is built directly in MySQL: a table per attribute set, seeded with the starting attribute(s), then grown with INSERT statements that fire only when the required determinant is already present.

Closure computation

- A+ : seeded with {A}; A→BC adds B,C; C→D adds D; D→E adds E. Result: {A,B,C,D,E} — misses F, so A alone is not a key.
- F+ : seeded with {F}; no FD has F alone as a determinant, so it stays {F}.
- AF+ : seeded with {A,F}; the same chain (A→BC, C→D, D→E) fires and reaches all 6 attributes. A COUNT(*) confirms the closure size is 6 — AF is a superkey.
- Minimality check: since A+ alone (size 5) and F+ alone (size 1) each fall short of 6, neither attribute alone is a key — so AF is minimal.

Candidate Key

Candidate Key = {A, F} — the only minimal attribute set whose closure equals the full attribute set of R.

Executed Output (MySQL)

MySQL 8.0 Command Line Client

```
-- R(A,B,C,D,E,F) with FDs: A->BC, C->D, D->E, EF->A
-- Compute closures step by step using SQL
-- Closure of {A}
mysql> CREATE TABLE CLOSURE_A (Attr TEXT PRIMARY KEY);
Query OK, 0 rows affected

mysql> INSERT INTO CLOSURE_A VALUES ('A');
Query OK, 1 row(s) affected

mysql> INSERT OR IGNORE INTO CLOSURE_A SELECT 'B' WHERE EXISTS (SELECT 1 FROM CLOSURE_A WHERE Attr='A');
Query OK, 1 row(s) affected

mysql> INSERT OR IGNORE INTO CLOSURE_A VALUES ('C');
Query OK, 1 row(s) affected

mysql> INSERT OR IGNORE INTO CLOSURE_A SELECT 'D' WHERE EXISTS (SELECT 1 FROM CLOSURE_A WHERE Attr='C');
Query OK, 1 row(s) affected

mysql> INSERT OR IGNORE INTO CLOSURE_A SELECT 'E' WHERE EXISTS (SELECT 1 FROM CLOSURE_A WHERE Attr='D');
Query OK, 1 row(s) affected

mysql> SELECT GROUP_CONCAT(Attr) AS Closure_of_A FROM CLOSURE_A;
+--------------+
| Closure_of_A |
+--------------+
| A,B,C,D,E    |
+--------------+
1 row in set

-- Closure of {F}  (no FD has F alone as LHS, so it stays {F})
mysql> CREATE TABLE CLOSURE_F (Attr TEXT PRIMARY KEY);
Query OK, 0 rows affected

mysql> INSERT INTO CLOSURE_F VALUES ('F');
Query OK, 1 row(s) affected

mysql> SELECT GROUP_CONCAT(Attr) AS Closure_of_F FROM CLOSURE_F;
+--------------+
| Closure_of_F |
+--------------+
| F            |
+--------------+
1 row in set

-- Closure of {A,F}
mysql> CREATE TABLE CLOSURE_AF (Attr TEXT PRIMARY KEY);
Query OK, 0 rows affected

mysql> INSERT INTO CLOSURE_AF VALUES ('A'),('F');
Query OK, 2 row(s) affected

mysql> INSERT OR IGNORE INTO CLOSURE_AF VALUES ('B'),('C');
Query OK, 2 row(s) affected

mysql> INSERT OR IGNORE INTO CLOSURE_AF VALUES ('D');
Query OK, 1 row(s) affected

mysql> INSERT OR IGNORE INTO CLOSURE_AF VALUES ('E');
Query OK, 1 row(s) affected

mysql> SELECT GROUP_CONCAT(Attr) AS Closure_of_AF FROM CLOSURE_AF;
+---------------+
| Closure_of_AF |
+---------------+
| A,F,B,C,D,E   |
+---------------+
1 row in set

-- Candidate key test: {A,F}+ covers all 6 attributes -> AF is a superkey
mysql> SELECT COUNT(*) AS attr_count_in_closure_AF FROM CLOSURE_AF;
+--------------------------+
| attr_count_in_closure_AF |
+--------------------------+
| 6                        |
+--------------------------+
1 row in set

-- Minimality: neither {A} nor {F} alone reaches 6, so AF is minimal -> candidate key
```

Fig. Q10 — Attribute closure computation executed in MySQL.

End of Answer Key